

1. 研究背景与关键问题

• 背景介绍

随着云存储服务的普及，用户希望将数据加密后存储在不可信的服务器上，同时又能高效地进行关键词检索。这一需求催生了**可搜索对称加密 (SSE)** 技术，其目标是在保证数据和查询隐私安全的前提下，实现对加密数据的高效搜索。

传统的SSE方案大多针对硬盘 (HDD) 的大块顺序访问优化，因此侧重于**局部性 (locality)** 的优化。但随着固态硬盘 (SSD) 逐渐成为主流存储介质，其性能更多地受“**页面访问数**”影响，而非连续内存访问数量。

• 关键问题

- 性能瓶颈转变**：现有SSE方案虽然在对称加密操作上足够高效，但在SSD环境下，由于大量随机访问页面的问题，实际性能严重受限。
- 设计指标不匹配**：传统的“局部性”指标 (locality) 无法准确反映SSD的实际读写性能，因此亟需新的设计目标，如**页面效率 (page efficiency)**。
- 数据分配挑战**：如何在保证数据隐私泄露最小化的前提下，高效地将变长数据“打包”到固定页面中，同时减少额外的存储开销和客户端存储 (stash) ？

2. 研究目的

• 主要目标

本文旨在构建一种**页面高效的SSE方案**，即在SSD上实现常数级页面访问 ($O(1)$) 和存储效率，同时满足安全性要求和最小的客户端存储负担。

• 具体目标

- 引入“页面效率”的概念，以反映SSD中页面访问数对整体性能的影响。
- 设计一种数据分配机制——**数据独立打包 (Data-Independent Packing, 简称DIP)**，使得数据在加密存储时，**其分布与查询无关**，从而避免泄露列表长度等信息。
- 利用一种基于最大流算法的分配策略，解决变长数据分配中的最优分配问题，并推广自经典的布谷鸟哈希，使其适用于变长数据分配。

3. 核心创新点

• 页面效率指标

提出并定义了针对SSD环境的页面效率指标，着重衡量查询时实际需要读取的页面数与明文数据所需页面数的比值。

• 数据独立打包 (DIP) 模型

构建了DIP模型，该模型通过**两哈希函数 (H1和H2)** 将每个关键词对应的数据列表随机分配到两个固定页面位置，再利用最大流算法对过载页面进行“边翻转”，实现最优分配，从而将未能分配到固定位置的数据尽可能地压缩到一个大小独立于数据库总量的客户端“stash”中。

• 优化分配算法

采用最大流算法对打包问题进行求解，通过将问题转化为图中边的定向问题，最小化各页面的溢出 (overflow)，证明了所获得的分配方案在满足容量约束下是最优的 (见文中定理4)。

• 理论与实践的结合

除了严格的理论证明外，还提供了详细的实验评估，展示了Tethys方案在SSD上低扩展因子 (少于3倍扩展) 下实现高效数据流传输的优势。

4. 文献相关研究时间轴

- 2014年
 - Cash和Tessaro证明了一个不可能性结果，指出在严格保护查询隐私的前提下，不可能同时兼顾存储效率、读取效率和局部性 (locality) 。（参见[CT14]）
 - 部分早期SSE方案（如[CJJ+14]）采用了**列表填充 (padding)** 的方法，但在小列表场景下导致存储膨胀严重。
- 2016年
 - Asharov等人提出了基于局部性优化的SSE方案 ([ANSS16])，引入了二维分配视角，虽然在局部性上取得进展，但读取效率仍存在对数级别的开销。
- 2018年
 - Demertzis等人提出了改进方案 ([DPP18])，在一定程度上降低了读取效率的对数因子。
- 2021年
 - 本文提出Tethys方案及其变体 (Pluto和Nilust)，首次在SSD环境下实现了同时具有常数级页面效率和存储效率的SSE方案。

5. 研究方法与分析（附表格说明）

- 方法概述

本文的方法主要分为两部分：

- 1. 构造DIP方案

- 利用两哈希函数确定每个关键词数据列表的两个候选页面位置。
 - 将每个关键词对应的变长数据列表视作多个“边”，构造类似布谷鸟哈希的图模型。
 - 通过最大流算法，寻找从过载页面到未满页面的翻转路径，最小化整体溢出量。
 - 最终未能放入固定页面的数据存储到客户端“stash”中。

- 2. 构造SSE方案

- 在DIP的基础上，构建SSE框架。
 - 利用安全伪随机函数 (PRF) 生成关键词的密钥和掩码，确保查询时数据分布和访问路径与真实数据无关。
 - 通过加密存储各页面（对应一个SSD页面）和查询时只访问固定的几个页面，达到页面效率常数级的目标。

- 表格分析：不同方案的比较

方案	客户端存储	页面成本	页面效率	存储效率	备注
Π_{bas} ([CJJ+14])	$O(1)$	$O(X \cdot p)$	$O(p)$	$O(1)$	采用填充方法，适合长列表但小列表浪费严重
Π_{pack} , Π_{2lev} ([CJJ+14])	$O(1)$	$O(X)$	$O(1)$	$O(p)$	小列表时存储膨胀问题明显

方案	客户端存储	页面成本	页面效率	存储效率	备注
1-Choice ([ANSS16])	$O(1)$	$\tilde{O}(\log N) \cdot X$	$\tilde{O}(\log N)$	$O(1)$	读取效率对数级别，实际表现受限
2-Choice ([ANSS16])	$O(1)$	$\tilde{O}(\log \log N) \cdot X$	$\tilde{O}(\log \log N)$	$O(1)$	改进方案，但仍非常数级别
Tethys (本文)	$O(p \cdot \omega(\log \lambda) / \log m)$	$2X + 1$	$O(1)$	$3 + \epsilon$	同时实现常数级页面效率和存储效率，性能领先SSD场景
Pluto (本文变体)	$O(p \cdot \omega(\log \lambda) / \log m)$	$X + 2$	$O(1)$	$3 + \epsilon$	提供了不同的存储与读取效率折中选择
Nilust (本文变体)	$O(p \cdot \omega(\log \lambda) / \log m)$	$2tX + 1$	$O(2t)$	$1 + (2/e)^{(t-1)}$	当桶的容量扩大为 $t \cdot p$ 时，存储效率接近理论最优值

表中 X 代表明文答案所占的页面数， ϵ 为任意小正数， t 为扩展参数。（数据摘自文献中的Table 1）

• **实验评估**

文章还通过在SSD上进行大量实验验证了所提方案的实际效果，结果表明在保持较小的密文膨胀因子下，Tethys方案能达到SSD原始吞吐量的一半左右，从而证明了理论与实践的一致性。

6. 主要结果与结论

• **理论结果**

- 构造了基于DIP的SSE方案Tethys，该方案实现了常数级别的页面效率（ $O(1)$ ）和存储效率（ $O(1)$ ），并能在严格的安全模型下保证最低限度的泄漏。
- 提出了一种基于最大流算法的最优分配方案，证明了在给定两个哈希函数的前提下，其分配方案能最小化溢出（stash中数据量），并给出了严格的stash大小上界证明（见定理5）。
- 进一步提出了两种变体（Pluto和Nilust），在存储效率和页面效率上提供了灵活的折中选择，特别是Nilust在桶扩展的情况下可以趋近信息论最优存储效率。

• **实验结果**

- 实验评估验证了理论预测：在大规模数据集上，stash大小与数据库规模无关，且页面读取数与明文数据所需页面数常数级相关。
- 在实际SSD设备上，Tethys方案显示出极佳的性能，能够大幅降低由于随机页面访问带来的性能损耗。

• **结论**

本文不仅在理论上为SSE方案设计提出了全新的数据分配模型和优化目标，还在实践中证明了其在SSD环境下的卓越性能，为今后基于固态存储介质的加密搜索系统设计提供了重要参考。

7. 延伸思考问题

1. 动态环境下的扩展

如何将本文提出的静态DIP方案扩展到支持动态数据更新（如插入、删除操作）的SSE方案，并在不牺牲页面效率 and 安全性前提下，保证动态更新的高效性？

2. 不同存储介质的适应性

除了SSD之外，针对新型存储设备（例如NVMe或新兴存储级内存），页面效率的衡量指标是否需要进一步调整？如何设计适应不同存储介质特性的SSE方案？

3. 跨领域应用可能性

DIP模型和基于最大流的分配策略能否推广应用到其他加密数据处理问题（如长度隐藏的ORAM、隐私保护的数据聚合）中，进而提高这些领域的系统效率？

“Once each list is assigned its two possible buckets, we are faced with two problems. The first problem is algorithmic: how should each list be split between its two destination buckets and the stash, so that the stash is as small as possible, subject to the constraint that the assignment is correct (all list elements are stored, no bucket receives more than p elements)? We prove that a simple max flow computation yields an optimal solution to this optimization problem. To see this, view buckets as nodes in a graph, with lists corresponding to edges between their two destination buckets, weighted by the size of the list. Intuitively, if we start from an arbitrary assignment of items to buckets, we want to find as many disjoint paths as possible going from overfull buckets to underfull buckets, so that we can “push” items along those paths. This is precisely what a max flow algorithm provides.” (Bossuat 等, 2021, p. 4) 一旦每个列表被分配了两个可能的桶，我们就面临着两个问题。第一个问题是算法：每个列表应该如何在两个目标桶和存储之间进行分割，以便在分配正确的约束下（所有列表元素都已存储，没有桶接收的元素超过 p 个），存储尽可能小？我们证明了一个简单的最大流计算可以得到这个优化问题的最优解。要看到这一点，请将bucket视为图中的节点，列表对应于其两个目标bucket之间的边，按列表的大小加权。直观地说，如果我们从任意将项目分配到桶开始，我们希望找到尽可能多的不相交路径，从满桶到欠桶，这样我们就可以沿着这些路径“推送”项目。这正是最大流算法所提供的。

这段话描述了 **数据分配到桶 (buckets)** 的核心问题，并提出了一种基于 **最大流 (max flow)** 计算的优化方案来解决它。下面我逐步解析它的含义：

1. 问题背景

在 TethysDIP 及其变体 (Pluto 等) 中，每个关键词对应的文档列表 (list) 会被分配到 **两个候选桶 (buckets)**，通过两次哈希 (H1 和 H2) 来决定其可能的存储位置。

但是，在完成初步分配后，会面临两个核心问题：

- **问题 1：**如何在这两个候选桶之间拆分这个列表，以尽可能减少 stash (溢出数据) ？
 - **问题 2：**如何确保每个桶的存储量不会超过上限 (p) ？
-

2. 用最大流 (Max Flow) 来求解

为了优化数据的分配方式，使 stash 最小，文章采用了 **最大流 (Max Flow)** 算法，并将问题建模为一个 **图 (graph)** 问题。

如何构造这个图？

- 桶 (buckets) → 视为图中的“节点”
- 关键词的文档列表 (lists) → 视为连接两个桶的“边”
 - 这些边的权重 (weight) 就是该关键词文档列表的大小

最大流的目标

- 在初始状态下, 某些桶可能会被分配过多数据 (超出容量 (p)), 而另一些桶可能未被完全填满。
- 目标是找到一组**无重叠 (disjoint) 路径**, 让数据可以从**过满的桶“流动”到未滿的桶**, 使得整体的存储分布更加均衡, 最终减少溢出的数据量 (stash) 。

3. 直观理解

你可以把这个问题想象成一个水管系统:

- 桶 = 水箱
- 列表 = 水流
- 最大流算法 = 重新调整水流的策略
 - 一开始, 一些水箱 (桶) 装满了水, 而另一些没有完全装满。
 - 目标是找到**最优的水流调整方式**, 让水从装满的水箱流向未滿的水箱, 而不让任何水溢出来 (即 stash 最小化) 。
 - 这正是最大流算法可以解决的问题。

4. 关键结论

- 通过构造上述图模型, **最大流算法可以计算出一个最优的分配方案, 使得 stash 最小。**
- 这个方法确保:
 1. **所有文档列表的元素都被正确存储** (不会丢失任何数据) 。
 2. **没有任何桶会超载** (每个桶存储的数据不会超过设定的上限 (p)) 。
 3. **stash 量最小**, 即未能被分配到桶的数据量尽可能少。

5. 总结

这段话的核心意思是:

1. 关键词的文档列表会被分配到两个桶, 但如何具体拆分它们是一个优化问题。
2. 这个优化问题可以建模成一个**最大流 (Max Flow) 问题**, 其中桶是图中的节点, 文档列表是加权边。
3. 通过运行最大流算法, 我们可以优化数据的存储方式, 使得溢出的数据量 (stash) 最小, 同时保证每个桶的存储量不超过上限。

这种方式提供了一种**数学上最优的分配策略**, 相比于简单的随机分配或贪心分配方式, 能够更好地减少溢出数据, 从而提高系统的查询效率和存储效率。

相关定义

读取模式 (Read Pattern) 指服务器在处理关键词 w 的搜索查询时, 访问加密数据库 EDB 的一系列内存位置集合, 记为 $\text{RdPat}(\tau, \text{EDB})$, 其中 τ 是与 w 关联的搜索令牌。

$$\text{RdPat}(\tau, \text{EDB}) \rightarrow m_1, \dots, m_h$$

局部性 (Locality) L 表示查询时 $\text{RdPat}(\tau, \text{EDB})$ 只需访问不超过 L 个连续的内存块。

读效率 (Read Efficiency) R 表示查询时所需的内存访问次数最多仅为存储明文答案的内存数的 R 倍。

$$|\text{RdPat}(\tau, \text{EDB})| \leq R \cdot P$$

P 是存储明文中与关键字 w 匹配的文档索引所需的**内存位置数量**

存储效率 (Storage Efficiency) S 表示加密数据库的存储成本最多是明文数据库的 S 倍。

$$|\text{EDB}| \leq S \cdot |\text{DB}|$$

页面模式 (Page Pattern) 指服务器在处理关键词 w 的搜索查询时引发的页面访问 $p_1, \dots, p_h$, 记为 $\text{RgPat}(\tau, \text{EDB})$

页面成本 (Page Cost) 表示 SSE 方案在查询时实际访问的页面数不会超过 $aX + b$, 其中 X 表示明文存储该查询结果所需的页面数。

$$|\text{RgPat}(\tau, \text{EDB})| \leq aX + b$$

页面效率 (Page Efficiency) 表示 SSE 方案在查询过程中读取的页面数量最多为明文存储所需页面数的 P 倍。

$$|\text{RgPat}(\tau, \text{EDB})| \leq P \cdot X$$

X 是以明文形式存储与关键字 w 匹配的文档索引**所需的页数**。

定理1 任何具有读取效率 R 和局部性 L 的 SSE 方案的页面效率最多为 $R+2L$ 。

SSE from Data-Independent Packing

“数据独立打包” (Data-Independent Packing, DIP)

多映射 $M = \{(K_i, (e_{i,1}, \dots, e_{i,\ell_i})) : 1 \leq i \leq k\}$

定义 3.1 DIP方案是一个三元组算法 (Size, Build, Lookup)

- $m \leftarrow \text{Size}(n)$ 根据输入值个数 n (所有列表元素总数?), 确定需要分配的桶的数量 m 。
- $(B, S) \leftarrow \text{Build}(M)$ 把多映射 M 构建成一个包含 m 个桶的数组 $B = (B[1], \dots, B[m])$ 和一个暂存区 S , 其中每个桶 $B[i]$ 的大小为 p , S 是另一个多映射

$$n = |M| = \sum_{1 \leq i \leq k} \ell_i$$

- $\mathcal{I} \leftarrow \text{Lookup}(m, K, \ell)$ 该算法接受桶数 m 、密钥 K 及其对应列表中的数据项数量 ℓ 作为输入, 返回一个包含桶索引的集合, 指出密钥 K 的数据项存储在哪些桶中。

正确性要求与密钥 K_i 关联的所有多映射值 $(e_{i,1}, \dots, e_{i,l_i})$ 要么在 $\text{Lookup}(m, K, \ell)$ 返回索引的桶中，要么在暂存区 S 中。

定义 3.2 (DIP方案的正确性)

$$\forall i \in [1, k] : M(K_i) \subseteq S \cup \bigcup_{j \in \text{Lookup}(m, K_i, \ell_i)} B[j].$$

DIP 的定义固有地强制数据独立性，因为Size 和 Lookup 都不依赖于 Build(M) 输入处的多映射 M ，而取决于它包含的值的数量。

DIP 相关效率指标的定义

查找效率 (Lookup Efficiency) L

$|\text{Lookup}(m, K, \ell)| \leq L \cdot x$ ，其中 x 是 $M(K)$ 需要的桶的最小数量

对于任意多映射中某个关键字所需的最少桶数 x ，Lookup 返回的桶数量不会超过 L 倍的 x 。

存储效率 (Storage Efficiency) E

$$m \leq E \cdot (n/p)$$

通过 Build 算法产生的桶数量 m 不超过理想桶数 n/p 的 E 倍，其中 n 是总数据量， p 是每个桶的容量。

暂存区大小 (Stash size) C

DIP 方案的暂存区中的数据项数量始终不会超过 C 个，无论输入的多映射 M 是怎样的。

为了构造一个高效的 SSE 方案，我们需要从以下几个角度着手：

- 通过减少桶的数量（存储效率），降低整体存储占用。
- 通过使查找操作返回的小桶集合（查找效率），在查询时减少页面读取。
- 同时控制客户端暂存区的大小。

如果只关注部分指标（如查找效率、存储效率或暂存区大小）中的任意两个，设计起来都比较简单——可以采用一些简单的策略满足两个指标，但往往会牺牲第三个指标。

总之，要同时在查找效率、存储效率和暂存区大小三个指标上都达到良好的性能，非常具有挑战性，需要更复杂的设计

SSE from Data-Independent Packing

Algorithm 2 TethysDIP

Build($m, M = \{(K_i, (e_{i,1}, \dots, e_{i,\ell_i})) : 1 \leq i \leq k\}$)

- 1: $B \leftarrow m$ empty buckets, $S \leftarrow$ empty stash
- 2: Create an oriented graph G with m vertices numbered $\{1, \dots, m\}$
- 3: **for all** values $e_{i,j}$ **do**
- 4: Create an oriented edge $(H_1(K_i), H_2(K_i))$ with label $e_{i,j}$
- 5: Add separate source vertex s and sink vertex t
- 6: **for all** vertex v **do**
- 7: Compute its outdegree d .
- 8: **if** $d > p$ **then**
- 9: Add $d - p$ edges from the source s to v
- 10: **else if** $d < p$ **then**
- 11: Add $p - d$ edges from v to the sink t
- 12: Compute a max flow from s to t
- 13: Flip every edge that carries flow
- 14: **for all** vertex $v \in \{1, \dots, m\}$ **do**
- 15: $B[v] \leftarrow \{e_{i,j} : \text{origin of edge } e_{i,j} \text{ is } v\}$
- 16: **for all** vertex $v \in \{1, \dots, m\}$ **do**
- 17: **if** $|B[v]| > p$ **then**
- 18: $|B[v]| - p$ values are moved from $B[v]$ to S
- 19: **return** (B, S)

Lookup($m, K, \ell \leq p$)

- 1: returns $\{H_1(K), H_2(K)\}$
-

Setup: 为每个关键字生成唯一且掩码化的令牌；利用 DIP 方案将关键字对应的文档索引分配到固定大小的桶中，并对溢出的部分进行存储；通过填充和加密（异或）操作来隐藏各桶内部真实数据量和列表长度信息。

Search: 客户端发送关键词令牌 $\rightarrow$ 服务器通过查表和 Lookup 得到相关加密桶 $\rightarrow$ 客户端解密这些桶并结合本地暂存区，还原完整结果列表。

效率分析

- 存储效率 $S+1$ （表 T 和 $|I|$ 个桶）
- 页面效率 $L+1$ （ L 是 D 的查找效率）
- 客户端存储 $C + O(1)$

安全性

- $\mathcal{L}_{\text{Setup}}(\text{DB}) = |\text{DB}| = N$ 初始化时暴露整个数据库的大小
- $\mathcal{L}_{\text{Search}}(\text{DB}, w_i) = (\ell_i, \text{sp})$ 查询时暴露关键词匹配的文档数和是否为重复查询（即 search pattern）

定理 2 (SSE 安全性) 设 D 为具有存储效率 S 、查找效率 L 和存储大小 C 的 DIP 方案。假设 Lookup 不进行任何页面访问，Enc 是 IND-CPA 安全加密方案，PRF 是安全伪随机函数。那么 SSE(D) 是一种具有存储效率 $S+1$ 、页面效率 $L+1$ 和客户端存储 $C+O(1)$ 的 L -自适应语义安全的 SSE 方案。

证明思路：使用一种叫做**模拟器 (simulator)** 的方法来证明。直观来说，如果一个“虚构的模拟器”仅根据泄露函数（比如列表长度、查询次数等）就能构造出一个让服务器无法区分的“假数据库”，那就说明真实数据库不会额外泄露其他信息。

👣 证明步骤：

1. 模拟 Setup：

- 模拟器知道 Setup 泄露的是数据库大小 N 。
- 它用 N 算出需要 m 个桶，然后为每个桶生成一些“伪数据”（任意值），并用 Enc 加密它们。因为 Enc 是 IND-CPA 安全的，所以这些伪密文对服务器来说看起来和真实密文没区别。
- 它还构造了辅助表 T ， T 的结构是：将随机值 κ 映射到另一组随机值 χ （模拟 $i \oplus m_i$ ）。因为实际方案中 i 被 m_i 随机掩码掩盖，所以这组数据本来就是看起来随机的。
- 最终，它构造出一个“看起来和真实一样”的模拟加密数据库 EDB。

2. 模拟查询 (Search)：

- 当查询 w_i 到来时，泄露函数告诉模拟器：该关键词匹配文档数为 ℓ_i 。
- 模拟器从表 T 中**随机选择一个尚未用过的** κ ，并用其构造令牌发送给服务器。
- 它构造出的查询令牌为： $\tau_i = (K_i, \ell \oplus T[K_i])$ 。
 - 注意：服务器在拿到 τ_i 后，会自己还原出 ℓ ，并使用 Lookup 取出桶索引 l 。
- 由于 K_i 是随机生成的，而 PRF 也是安全的，所以服务器无法区分模拟器生成的 K_i 和真实 PRF 的输出。
- 同样地，由于 i 被 XOR 加密，服务器也看不出其真实值。

✅ 模拟器的安全性保证：

- 桶是随机密文 $\rightarrow$ IND-CPA 加密保证不可区分；
- T 表中的 $i \oplus m_i \rightarrow$ 看起来就是随机值；
- K_i 是伪随机的 $\rightarrow$ PRF 安全保证不可区分；

所以，模拟器模拟出的整个 Setup 和 Search 行为，服务器看不出与真实世界的区别，安全性成立。

✅ 总结一句话：

这一定理通过构造一个模拟器，证明了只要 DIP 的数据独立性和随机性安全性成立，整个 SSE 系统就可以在标准泄露模型下达到语义安全，且拥有良好的存储和页效率。安全性成立的关键，是 DIP 的**数据独立性**和**不可重试性**。

Efficient Data-Independent Packing

为了简单起见，我们将每个密钥的值限制为最多 p （一页），即 $i \leq p$ 。

算法利用两个哈希函数 H_1 和 H_2 来为每个键 K_i 确定两个候选桶。具体地，

- H_1 将键映射到前半的桶（编号 1 到 $m/2$ ），
 - H_2 将键映射到后半的桶（编号 $m/2+1$ 到 m ）。
- 这种设计（两个哈希函数且范围不重叠）在后续分析中有助于控制备用存储区的大小（stash size）。

对于每个键 K_i 的值，可以灵活地分配到两个候选桶中：

- 可以有 a 个值放到 $H_1(K_i)$ 对应的桶，
 - b 个值放到 $H_2(K_i)$ 对应的桶，
 - 如果两个桶都没有足够空间，则剩下的 c 个值存入备用存储区 (stash)。
- 总体要求满足 $a + b + c = \ell_i$ 且每个桶中存放的值总数不超过桶容量 p 。

算法描述

1. 构造图模型：

- 将所有桶视为图中的顶点。
- 对于每个值 $e_{i,j}$ (属于关键字 K_i)，根据两个候选桶 $H_1(K_i)$ 和 $H_2(K_i)$ 画一条边。
- 可能存在多条边连接同一对桶。
- 初始时，这些边被任意定向，定向的含义是：每条边的起点桶将负责存储该边对应的值；因此，一个桶的负载等于它作为起点的边的数量。

2. 利用边翻转来平衡负载：

- 如果存在一条有向路径，翻转这条路径上所有边的方向：
 - 翻转后，路径起点桶的负载减少 1，终点桶的负载增加 1，而路径中间的桶负载保持不变。
- 这种操作的直观意义是：将部分数据从过载的桶“转移”到欠载的桶，从而使得整体分配更加均衡，减少因某个桶超出容量 pp 而需要放入备用区的数据量。

3. 最大流算法的应用：

- 为了在整个图中找到尽可能多的不相交路径（即这些路径之间互不干扰），从过满（超载）的桶到未满（欠载）的桶，算法调用了最大流算法。

这句话的意思是，算法试图在图中找到一组路径，这些路径满足两个条件：

1. 每条路径都从一个负载过高（超出容量）的桶出发，连接到一个负载较低（未满）的桶。
2. 不同路径之间没有共享任何边或顶点，也就是说，它们是“互不干扰”的，不会互相影响。

换句话说，这组路径是“边不相交”或“顶点不相交”的，每条路径都可以独立地翻转边的方向，从而将数据从超载的桶转移到欠载的桶，而不干扰其他路径的调整。

- 最大流算法帮助确定一组“最优”的路径集合，翻转这些路径后可以使得更多的值被成功放入桶中，而不是被送入 stash。

4. 最终分配和溢出处理：

- 翻转完成后，每个值按照它所在边的起点被分配到对应的桶中。
- 如果某个桶接收的值超过了容量 pp （即负载过高），多余的部分会被放到备用区 (stash)。

下面我逐步解释一下伪代码的每个步骤和它们的作用：

Build 算法

输入：

- 桶的数量 m
- 多映射 $M = \{(K_i, (e_{i,1}, \dots, e_{i,\ell_i})) : 1 \leq i \leq k\}$, 其中每个键 K_i 对应 ℓ_i 个值
- 此处假设每个键的 $\ell_i \leq p$ (即每个键对应的值不超过一个桶的容量)

输出:

- 一个字典 B , 其中 $B[v]$ 存放分配到桶 v 的值
- 一个备用存储区 (stash) S , 用于存放那些因桶容量限制而无法分配的多余值

步骤说明:

1. 初始化桶和暂存区:

1: $B \leftarrow$ 创建 m 个空桶;
同时初始化 S 为空备用区。

2. 构建图 G :

2: 创建一个有向图 G , 图中有 m 个顶点, 对应桶编号 $1, \dots, m$ 。

3. 为每个值建立边:

3-4: 对于多映射中每个键 K_i 的每个值 $e_{i,j}$, 根据两个哈希函数 H_1 和 H_2 计算出两个候选桶 $H_1(K_i)$ 和 $H_2(K_i)$ 。

然后, 画一条有向边, 从 $H_1(K_i)$ 指向 $H_2(K_i)$, 边上标记 $e_{i,j}$ 。

这表示初始时, 值 $e_{i,j}$ 默认分配到桶 $H_1(K_i)$ (即边的起点)。

4. 添加源点和汇点构造流网络:

5: 添加一个单独的源点 s 和汇点 t 。

这种操作是为后续构造流网络做准备, 用于调整桶之间的负载。

5. 根据桶负载调整边:

6-11: 对图中每个顶点 v (即每个桶) :

- 7: 计算该顶点的出度 d (即当前桶中分配的值的数量)。
- 8-9: 如果 $d > p$ (桶中值超出容量), 则从源点 s 向桶 v 添加 $d - p$ 条边;
- 10-11: 如果 $d < p$ (桶中值未满足), 则从桶 v 向汇点 t 添加 $p - d$ 条边。

这种构造将每个桶与源或汇连接, 反映了每个桶与理想容量 p 的差距。

6. 求最大流并翻转边:

12: 在构造好的流网络中, 从 s 到 t 计算最大流。

13: 对所有携带流量的边, 翻转它们的方向。

翻转边的直观意义是: 如果存在一条路径能够把超载桶 (从 s 连出的边) 与未满足桶 (连接到 t 的边) 连接起来, 则通过翻转路径上所有边, 可以将一个值从负载过高的桶“转移”到负载不足的桶, 从而使原本超出的负载得到调整。

7. 构造最终的桶内容:

14-15: 遍历所有桶 (顶点 $v = 1, \dots, m$), 将所有“出发” (即边的起点) 于 v 的边上的值 $e_{i,j}$ 收集到 $B[v]$ 中。

这时, 每个桶中存放的值就是最终分配到该桶的值。

8. 处理溢出到备用区 (stash) :

16-18: 再次遍历每个桶 v :

- 如果桶 v 中的值数量超过容量 p (即 $|B[v]| > p$), 就将超出 p 的那部分值移到备用区 S 。
- 这样可以确保每个桶中最多有 p 个值。

9. 返回结果:

19: 返回最终的桶分配 B 和备用区 S 。

Lookup 算法

功能：对于给定的键 K 和其对应的值数 $\ell \leq p$ ，直接返回该键对应的两个候选桶的索引，即 $\{H_1(K), H_2(K)\}$ 。

说明：

Lookup 非常简单，因为 DIP 方案中，每个键总是与两个候选桶关联；在查询时只需要知道这两个桶的编号，即可定位到存储该键值的桶（以及备用区中的溢出部分由客户端维护）。

总结

- **Build 算法** 通过构造一个类似布谷鸟哈希的图，将每个值用一条边连接到两个候选桶。
- 通过将桶与源点、汇点连接，构造流网络，并利用最大流算法找出可以翻转的边路径，调整各桶的负载。
- 翻转后的边决定了最终每个值应分配到哪个桶；如果某个桶的值数超出容量 pp ，多余部分进入备用区 S 。
- **Lookup 算法** 则简单地返回一个键的两个候选桶索引，供搜索时使用。

这种设计的核心在于利用图论和最大流方法，优化全局的分配策略，使得在桶容量限制下，尽可能多的值能被合理分配，从而最小化备用区的使用。

效率分析

TethysDIP分配了 $m = (2 + \varepsilon)n/p$ 桶的总数，因此有**存储效率** $2 + \varepsilon = O(1)$

查找效率为 $2 = O(1)$

暂存区大小 $p \cdot \omega(\log \lambda) / \log m$ 是足够的，暂存区大小不会随着多映射 M 的大小而增加。

怎么处理任意长度的列表？

拆分成大小为 p 的子列表，再用 TethysDIP 处理即可

为了区分原来一个键拆分后的多个子列表，每个子列表关联一个新键，记为 $K_i || j$ （其中 j 表示第 j 个子列表）。这样，新构造的多映射 M' 中，每个键对应的值列表长度都不超过 p 。

然后，对新构造的 M' 应用 TethysDIP 算法，因为在 M' 中每个列表都满足 $\ell \leq p$ 。

当需要查询原始键 K_i 时，只需对所有新键 $K_i || j$ ($1 \leq j \leq \lceil \ell_i/p \rceil$) 分别执行查找，最终合并结果即可还原出原始的值列表。

其效率与正确性不变

我们可以用TethysDIP实例化第3.1节中给出的框架。这产生了一个SSE方案

$\text{Tethys} := \text{SSE}(\text{TethysDIP})$

定理3 假设 Enc 是 IND-CPA 安全加密方案，PRF 是安全伪随机函数， H_1 、 H_2 是随机神谕。然后，Tethys 是一种 L - 自适应语义安全的 SSE 方案，具有存储效率 $O(1)$ 、页面效率 $O(2)$ 和客户端存储 $O(p \cdot \omega(\log \lambda) / \log m)$ 。

暂存区大小分析

最优性

定理4 (Optimality of TethysDIP)

给定一个无向图 G ，令 D 为 TethysDIP 算法在输入 G 后输出的有向图。则对于所有由 G 导出的有向图中， D 所对应的溢出量 $\text{overflow}(D)$ 是最小的。

这里的“溢出量”定义为：

$$\text{overflow}(D) = \sum_{v \in V} \max(0, \text{out}(v) - p)$$

其中 V 表示所有桶（图中的顶点）， $\text{out}(v)$ 表示顶点 v 的出度，即分配到该桶的数据项数，而 p 是桶的容量上限。换句话说，溢出量就是所有桶中超出容量 p 部分数据项的总数，这些数据项最终需要存到 stash 中。

证明思路概述

证明利用了**最大流—最小割定理**，其主要思路可以分为以下几个步骤：

1. 构造流网络

- 首先，将问题转化为一个流网络问题。这里，每个桶（顶点）都有一个容量限制（ p ）（即允许存储的最大数据项数）。
- 每条边（对应一个键及其数据项）有两个可能的去向，分别代表将该数据项放入 $H_1(K)$ 或 $H_2(K)$ 对应的桶中。这个过程中可以看作在两个桶之间进行选择。

2. 最大流与最小割的关联

- 利用最大流算法，寻找一种数据流（即一种分配方案），使得尽可能多的数据项都被“送入”桶中而不造成溢出。
- 根据最大流—最小割定理，最大流的值等于最小割的容量。这里的最小割实际上划分了图的顶点集合 V 成两个部分：集合 S 和集合 T （其中 S 包含源， T 包含汇）。
- 在这个划分中，最小割容量就对应着必须溢出到 stash 中的数据项数量。

3. 翻转流向与割集性质

- 证明中采用了在算法第13行中翻转边方向的操作。翻转后，证明了不存在从集合 S 指向集合 T 的边，即不存在可以再“调配”数据项以减小溢出量的可能性。
- 进一步证明，所有溢出的数据项都位于集合 S 内，而集合 S 中的每个桶已经达到或超过其容量 p （即已经满载或超载），因此无法再通过调整内部边的方向来减少溢出量。

4. 最优性结论

- 由于任何在 S 内部的边调整只能导致溢出量增加或不变，而没有额外的边可以从 S 流向 T 以减轻 S 的负载，因此证明了 TethysDIP 输出的有向图 D 的溢出量已经达到最小值。
- 换句话说，TethysDIP 在所有可能的边定向方案中找到了一个全局最优的方案，使得 stash 中的数据（即溢出量）不可再减少。

证明中的关键点

• 最大流—最小割定理的应用：

证明利用这一经典定理，将数据项分配问题等价转换为流网络中的最大流问题，而最小割恰好表示了无论如何调整，仍然必须溢出的最小数据项总数。

- **集合 S 与 T 的划分：**

通过将顶点划分为两个集合，并证明所有溢出项都集中在 S 中，再加上 S 内部桶均已满或超载，使得没有边能把 S 内的溢出数据重新分配到 T 中，从而证明了全局最优性。

- **翻转边方向的策略：**

翻转边的操作确保了边的方向已经经过调整，使得没有边能够进一步从 S 流向 T，这实际上锁定了当前的分配状态，从而保证无法再通过重新定向边来降低溢出量。

总结

定理4证明了 TethysDIP 算法在解决分配问题时具有全局最优性，即在所有合法的分配方式中，它输出的方案使得桶中超出容量 p 的溢出量（也就是需要放入 stash 的数据项数）最小。证明的核心在于将问题建模为流网络，应用最大流—最小割定理，通过构造合适的集合 S 与 T 以及翻转边方向的技巧，说明了任何可能的调整都不可能进一步减少溢出量，从而确立了 TethysDIP 的最优性。

这种证明方法不仅展示了算法内部的精妙设计，同时也说明了这种方法对哈希函数的选择、桶的数量以及初始图的方向都具有普适性，无论如何，算法都能达到最小溢出量的目标。

Stash的大小上限

定理5 (Stash size bound)

在给定参数条件下（例如：设 $\varepsilon > 0$ 是任意常数，桶数 $m = (2 + \varepsilon)n/p$ ，列表中每个键映射的值不超过 p ，以及 stash 大小 $s = n^{o(1)}$ ，对于任意满足这些条件的多映射 L 和哈希函数 $H = (H_1, H_2)$ ，有

$$\Pr[\text{Fail}_{m,p,s}(L, H)] = \mathcal{O}\left(p \cdot m^{-s/(2p)}\right)$$

换句话说，只需要大约 $\omega(\log \lambda) / \log m$ 页的 stash（当 $m = \lambda^{\Omega(1)}$ 时，只需 $\omega(1)$ 页）就能保证 TethysDIP 失败的概率在密码学意义下是可忽略的。

- **参数说明**

- 向量 L 表示多映射中各列表的长度，要求每个列表最多 pp 个值且总和为 nn ；
- m 为桶的数量，按 $m = (2 + \varepsilon)n/p$ 来选取；
- 失败事件 $\text{Fail}_{m,p,s}(L, H)$ 表示无法将图中边定向，使得每个桶溢出（即超出 p 的部分）小于 s 的概率。

- **核心结论**

在满足参数条件下，stash 的大小不会随 n 增长，意味着客户端的存储要求可以保持常量级或仅随参数 λ 对数级增长，而不会与整个数据库规模挂钩。

定理证明的高层思路

证明定理5的思路大体分为以下几个步骤：

1. 凸性与期望上界

- 证明对任意列表长度分布，其 stash 大小的期望上界可以被一个特殊情况（即所有列表长度均为 p ）所控制，最多只会多项式因子上有所不同。
- 这里用到了一个**凸性论证**：证明最小 stash 大小作为图的函数是 Schur-凸的，这意味着在“最不利”情形（所有列表长度都为 p ）下，其期望值是最大的（或近似最大的）。

2. 等价于带 stash 的布谷鸟哈希问题

- 在“最不利”情况下，问题可以转化为经典的布谷鸟哈希问题（cuckoo hashing with a stash），这使得可以利用已有关于布谷鸟哈希的分析结果（例如 [Wie17] 中的结果）来给出 stash 大小的尾界（tail bound）。

3. 延展凸性论证得到尾界

- 通过扩展上述凸性论证，并结合问题的具体结构，推导出 stash 大小的尾界，最终证明失败的概率为

$$\mathcal{O}\left(p \cdot m^{-s/(2p)}\right)$$

从而说明只需要非常少量的 stash 空间（不随 n 增长）就能使失败概率在密码学上变得可忽略。

5. 总结

- **安全性与存储成本**

该定理和证明说明，在 TethysDIP 算法中，只需要使用 $m = \mathcal{O}(n)$ 个桶和一个与数据库规模无关的 stash（溢出区域）大小，就可以确保整个构造失败的概率在密码学意义上是微乎其微的。这对实际应用非常重要，因为 stash 存储在客户端，而客户端的存储资源通常有限。

- **与布谷鸟哈希的联系**

TethysDIP 的问题设置包含了布谷鸟哈希作为特例，因此使用 $2+\epsilon$ 的存储效率因子是最优的，也是必要的。这一参数选择避免了图中发生相变，从而保证系统在各项参数均处于“安全区”。

- **证明思路的关键**

证明通过凸性和主要化（majorization）技巧，将一般情况下的分析归约到最坏情形，再利用已知的布谷鸟哈希结果给出尾界，最终确保了 stash 的大小上界不会随着数据库规模 n 的增长而增加，同时失败概率下降。